

CAPSTONE PROJECT

Booking + Cancellation → Settlement

Agentic AWS · Day 12 · 4 hours

Build a real-time agentic pipeline that handles BOTH airline use cases from the glossary — booking-to-settlement AND cancellation-to-settlement — through one event-type-routed pipeline. Ingest from GDS, enrich via Bedrock, score via SageMaker, write the settlement delta. You demo one path live; the repo must support both. Pass at 22/30.

1. The business problem

Accelya's settlement engine reconciles money between airlines, agents and intermediaries. Two events drive that engine:

- Booking (Use Case 1): a PNR is created, ticket issued, agent owes the airline money. Settlement direction: agent → airline (positive delta).
- Cancellation (Use Case 2): ticket is cancelled, refund calculated against fare rules. Settlement direction: airline → agent (negative delta minus penalty).

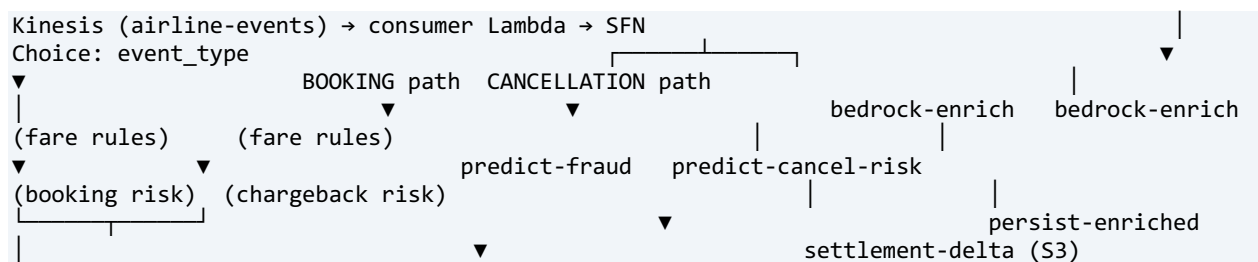
Today both flows live in batch jobs that run overnight and hand-built rules that don't see the passenger's history. Your capstone replaces them with one streaming pipeline that routes on `event_type`, enriches with Bedrock, scores risk with a SageMaker model, and writes settlement deltas in seconds.

In airline-glossary terms

- Booking event carries: PNR, Fare Class, OD, Fare Break, Ticket Number, `fare_amount`, `payment_method`
- Cancellation event carries: PNR, Cancellation Event ID, `cancel_reason_code`, Before/After Departure flag, No-Show flag
- Both routed through the same Kinesis stream; pipeline dispatches on `event_type` ("booking" | "cancellation")

2. The pipeline (one stream, two event types)

The Day-7 stream `cancellation-events-<learner>` is renamed to `airline-events-<learner>` and carries both event types. The Step Functions state machine routes on `event_type` early:



Critical design choice: ONE SageMaker endpoint, TWO prediction modes. The Day-9 endpoint is reused for both (booking-fraud vs cancel-risk) — feature engineering inside the wrapper Lambda decides which signature to send. This is the trade-off you defend in your demo (\$9 dim 10).

2.1 Stage-by-stage

Stage	Day source	What it does (both paths)
Producer Lambda	D7	Receives booking OR cancellation envelope, normalises, writes to Kinesis with PNR hash partition key
Consumer Lambda	D8	THIN — claims via DDB idempotency, starts SFN with correlation_id, traceHeader propagated
State Machine	D8/D9 ext.	Choice on event_type → BookingFlow OR CancellationFlow (Parallel branches per path)
Bedrock enrich	D8	Sampled 1-in-10. Booking: explain fare rules + payment terms. Cancellation: refund tier + No-Show reasoning.
Predict wrapper	D9	Booking: fraud_score (high fare_amount + new agent + odd OD). Cancellation: chargeback_risk (existing v1.2.3 model).
Persist enriched	D8	gzip JSONL to s3://.../enriched/<learner>/event_type=<booking cancellation>/ingest_date=.../
Settlement output	Capstone	On high-confidence: emit settlement_delta to s3://.../settlement/<learner>/event_type=.../ — Accelya reconciliation reads from here

2.2 Why this fits the 10-Lambda cap

No new Lambdas are added in the capstone. The 10 existing Lambdas handle both flows because the routing happens INSIDE the state machine (Choice state), not at the Lambda layer. Booking and cancellation share the same producer, consumer, bedrock-enrich, predict-wrapper, and persist Lambdas.

2.3 Observability + security (D10) — applies to both paths

- Powertools Logger + Metrics + Tracer on all 10 Lambdas; correlation_id propagated end-to-end
- Dashboard adds a `event\_type` dimension to RecordsOk metric → separate widgets for booking vs cancellation throughput
- 3 alarms wired to SNS (iterator-age, bedrock-throttle, confidence-drift via ANOMALY_DETECTION_BAND)
- Cost Anomaly Detection monitor (reuses Default-Services-Monitor)
- Tightened IAM policy (Access Analyzer green; zero wildcard Resource on write/modify verbs)

2.4 Deploy story (D11) — applies to both paths

- Single SAM template covers the whole stack
- `.gitlab-ci.yml` integration-test stage submits ONE booking event + ONE cancellation event and asserts both reach `settlement/` with correct deltas
- OIDC trust; OSS scanners (bandit + semgrep + tfint) gate the deploy stage

3. The 4-hour Day 12

Hour	Activity	Deliverable
1	Briefing + design — 15 min brief, 15 min individual design (paper + Cascade, no AWS calls), 15 min Cascade-assisted peer review, 15 min trainer reference walk-through. Design must show the event_type Choice point.	design.md committed with annotated architecture diagram
2	Build Sprint 1: Wire — `sam deploy` from D11 template; one BOOKING event + one CANCELLATION event both run end-to-end successfully	2 successful SFN executions (one per event_type)
3	Build Sprint 2: Harden — D10 dashboards live; deliberate failure injection (FIS ≤20 min on either path); Cascade as diagnostic agent for recovery	Failure injected + recovered, dashboard screenshot showing which path took the hit
4	Demo (5 min each) + rubric scoring + retrospective + close. You pick ONE path to demo live; the other must show successful execution in CW logs.	5-min demo recording + 1-page reflection

4. Rubric (10 dims × 3 = 30; pass ≥ 22)

Each dimension scored 0 / 1 / 2 / 3. Evidence column is what the trainer checks. Both event_types are evaluated — a pipeline that only handles booking OR only cancellation caps at dim 1 = 1.

#	Dimension	Evidence the trainer checks
1	Pipeline runs end-to-end (both paths)	One booking + one cancellation SFN execution each reach SUCCEEDED during demo window
2	Correlation IDs propagated	Same UUID across producer / SFN execution name / bedrock_enrich / predict / persist for one record on EITHER path
3	Bedrock observable	Dashboard shows AWS/Bedrock InvocationLatency + InvocationThrottles + InputTokenCount; alarm wired
4	Idempotency demonstrable	Replay same event_id (either path) → no duplicate in enriched/ or settlement/
5	SAM deploys cleanly in CI	GitLab pipeline green; sandbox stack created from MR; integration test exercises BOTH paths
6	One injected failure recovered	FIS scenario (Lambda throttle / Bedrock denied / SFN timeout) recovered end-to-end during Hour 3
7	Audit trail present	/reviews/ folder with HITL_REVIEW.md per artefact; design.md committed; settlement deltas verifiable per event_type
8	Cost ceiling honoured	Athena workgroup quota set; Bedrock <50 invocations across both paths; sandbox endpoint deleted EOD
9	MR shows Cascade review	At least one MR comment from running Cascade review prompt against the diff
10	Demo articulates one trade-off	Verbal: name the one-endpoint-two-prediction-modes choice or another and what alternative you rejected

5. Inputs and outputs

5.1 Booking envelope (Use Case 1 input)

```
{  "event_id":      "bkg-2026-05-21-001",  "event_type":    "booking",  "event_time":    "2026-05-21T07:30:00+00:00",  "airline_code":  "AI",  "source_system": "GDS",  "pnr":          "ABC123",  "booking_date":  "2026-05-21",  "correlation_id": "bkg-2026-05-21-001",  "payload": {    "pnr":          "ABC123",    "fare_class":    "Saver",    "origin_destination": "BOM-LHR",    "fare_break":    "BOM-DEL-LHR",    "ticket_number":  "098-1234567890",    "fare_amount":    55000.0,    "currency":       "INR",    "agent_code":     "AGT-IN-4421",    "payment_method": "card",    "passenger_count": 1  } }
```

5.2 Cancellation envelope (Use Case 2 input)

```
{  "event_id":      "cxl-2026-05-22-001",  "event_type":    "cancellation",  "event_time":    "2026-05-22T09:15:00+00:00",  "airline_code":  "AI",  "source_system": "GDS",  "pnr":          "ABC123",  "booking_date":  "2026-05-21",  "correlation_id": "cxl-2026-05-22-001",  "payload": {    "pnr":          "ABC123",    "fare_class":    "Saver",    "origin_destination": "BOM-LHR",    "fare_break":    "BOM-DEL-LHR",    "ticket_number":    "098-1234567890",    "cancellation_event_id": "CXL-2026-001",    "cancel_reason_code": "VOL",    "before_or_after_dep": "BEFORE",    "no_show":          false,    "fare_amount":      55000.0,    "refund_lag_hours":  2.5,    "prior_cancels":    0  } }
```

5.3 Booking enriched output (Use Case 1 persisted)

```
{  "event_id":      "bkg-2026-05-21-001",  "event_type":    "booking",  "correlation_id": "bkg-2026-05-21-001",  "pnr":          "ABC123",  "fare_class":    "Saver",  "ticket_number": "098-1234567890",  "fraud_score":    0.12,  "confidence":    0.93,  "model_version": "v1.2.3",  "low_confidence": false,  "bedrock_narrative": "Saver class one-way BOM-LHR; agent AGT-IN-4421 has clean 90-day history. Payment via card; settlement T+2.",  "settlement_delta": 55000.0,  "decided_at":    "2026-05-21T07:30:04+00:00" }
```

5.4 Cancellation enriched output (Use Case 2 persisted)

```
{  "event_id":      "cxl-2026-05-22-001",  "event_type":    "cancellation",  "correlation_id": "cxl-2026-05-22-001",  "pnr":          "ABC123",  "fare_class":    "Saver",  "refund_amount":  52000.0,  "penalty_applied": 3000.0,  "risk_score":    0.18,  "confidence":    0.91,  "model_version": "v1.2.3",  "low_confidence": false,  "bedrock_narrative": "Saver before departure: ₹3000 penalty; eligible for refund within 7 days.",  "settlement_delta": -52000.0,  "decided_at":    "2026-05-22T09:15:04+00:00" }
```

6. What is NOT in scope

- Actual money movement — `settlement\_delta` is a record, NOT a payment instruction
- Real-time integration with live Accelya systems — synthetic data only
- New ML model training — reuse cancel-risk-v1.2.3 from D9 for BOTH prediction modes (feature engineering inside the wrapper)
- Multiple SageMaker endpoints — sandbox cap = 1; do not deploy a separate booking-fraud endpoint
- Bedrock Agents / Knowledge Bases / Studio — sandbox-forbidden
- Anything outside `ap-south-1` except Cost Anomaly Detection (us-east-1, global service)
- Search / Offer / PNR-creation flows from the glossary — capstone starts at the ALREADY-CREATED ticket and processes downstream events only

7. Deliverables (one GitLab repo)

1. `infra/template.yaml` — SAM template covering all 10 Lambdas, Step Functions (with event_type Choice), DDB, SQS, Kinesis, dashboard, 3 alarms, SNS topic
2. `src/\*/handler.py` — 10 Lambdas, all powertools-instrumented with correlation_id propagation; predict wrapper handles BOTH booking-fraud and cancel-risk modes
3. `prompts/v15\_capstone.txt` — 15-layer prompt (after D11's DEPLOY CONTEXT)
4. `reviews/` — HITL_REVIEW.md per artefact (9-dim D11 rubric)
5. `design.md` — one-page architecture diagram showing the event_type Choice + 3 trade-offs explicitly named (one-endpoint-two-modes is one of them)
6. `.gitlab-ci.yml` — 4-stage pipeline (lint → test → sam-deploy-sandbox → integration-test); integration-test sends ONE booking + ONE cancellation event and asserts both settlement deltas
7. `tests/integration\_test.py` — pytest that exercises both paths end-to-end
8. 5-min demo recording (one path live, the other shown via CW logs) + 1-page reflection on what you'd do differently

8. End-of-capstone cleanup (non-negotiable)

Sandbox resets overnight, but two resources have caps that affect other learners:

```
bash starter-repo/scripts/bootstrap_day10.sh <your-name> --teardown sam delete --stack-name
accelya-capstone-<your-name> --region ap-south-1
```

Verify zero endpoints + zero stacks before you leave the room. The trainer's final check is this command, run on every learner's account.